



Comparaison expérimentale des algorithmes de chiffrement symétrique

Débit, stabilité statistique et diffusion cryptographique sur architectures x86 et ARM

UNIVERSITÉ TÉLUQ

INF1430 – TN4 – Projet de fin d'études

Travail présenté à Monsieur Habib Louafi

Par Melissa Moya

Numéro d'étudiant : 24332062

Remis le 3 août 2026



Dépôt du projet : github.com/moyamelissa/INF1430-Comparaison-Chiffrement-Symetrique

Table des matières

Introduction	3
Problématique	4
Architecture du système.....	5
Protocole expérimental	7
Matrice expérimentale	7
Mesure du débit.....	8
Intervalle de confiance à 95%	8
Effet d’avalanche	9
Reproductibilité entre plateformes	9
Validation fonctionnelle (KAT)	9
Sources normatives	10
Deux profils d’exécution	10
Intégrité des vecteurs externes	11
Bilan de l’exécution	11
Résultats et analyse	12
Débit de chiffrement selon la plateforme	12
Stabilité statistique des mesures	17
Effet d’avalanche.....	19
Modes de chiffrement.....	21
Vue synthétique multicritère	23
Synthèse.....	25
Validation des hypothèses initiales	26
Recommandations selon le contexte de déploiement.....	26
Perspectives et risques émergents.....	27
Limites de l’étude	28
Conclusion générale	29
Références	29
Annexes	31

Introduction

La sécurité de l'information constitue une exigence fondamentale des systèmes informatiques modernes, tant pour les communications réseau que pour le stockage de données et les environnements embarqués à ressources limitées. Dans ce contexte, le chiffrement symétrique reste une famille de solutions privilégiée en raison de son efficacité opérationnelle. Cette efficacité ne peut toutefois pas être jugée sur la seule base de la robustesse théorique d'un algorithme, puisque la performance réellement observée dépend tout autant de facteurs d'architecture, notamment le type de processeur, la hiérarchie mémoire et la présence d'instructions cryptographiques dédiées. Cette articulation entre propriétés cryptographiques et contraintes d'exécution justifie une évaluation expérimentale rigoureuse dans un cadre comparable.

Le présent rapport examine cette articulation au moyen d'une comparaison expérimentale de cinq algorithmes de chiffrement symétrique, AES (Advanced Encryption Standard), DES (Data Encryption Standard), 3DES (Triple DES), Twofish et ChaCha20, sur deux plateformes contrastées. La première est une machine x86 sous Windows équipée d'un Intel Core i5-10300H avec AES-NI (Advanced Encryption Standard New Instructions). La seconde est un Raspberry Pi 4 à architecture ARM (Advanced RISC Machine) Cortex-A72, où l'accélération AES n'est pas explicitement exploitée dans ce protocole expérimental. Les implémentations reposent principalement sur PyCryptodome pour AES, DES, 3DES et ChaCha20, tandis que Twofish est pris en charge par un module externe dédié.

Cette comparaison a été conduite selon une approche SDLC (Software Development Life Cycle), structurée en quatre phases alignées sur les activités fondamentales décrites par Sommerville (2016, p. 45) soit la spécification, le développement, la validation et l'évolution, puis adaptées au contexte d'un projet de recherche expérimentale. La première phase, correspondant à la spécification, a couvert l'analyse des fondements cryptographiques et la planification expérimentale. La deuxième, relevant du développement, a formalisé la

conception, l'architecture des composantes logicielles et l'organisation du dépôt de code, ainsi que la préparation de l'environnement d'essai. La troisième, centrée sur la validation, a établi une base de fiabilité avant l'analyse finale, en combinant la validation fonctionnelle par KAT (Known Answer Tests), une première campagne de mesures avec estimation de la stabilité par intervalle de confiance à 95 % (IC95), et la vérification d'intégrité SHA-256 des vecteurs externes. La quatrième phase a consolidé cette campagne sur une base de données propre, en réexécutant les mesures de débit, de stabilité statistique et d'effet d'avalanche, puis en régénérant les graphiques et tableaux finaux pour l'analyse critique. Cette phase a également introduit une chaîne d'intégration continue automatisant les tests avec suivi de couverture ainsi que les audits de sécurité des dépendances, une infrastructure qui rejoint directement la dimension d'évolution décrite par Sommerville (2016, pp. 60-61) pour qui la capacité à faire évoluer un système en toute sécurité repose sur des mécanismes de validation continue plutôt que sur une vérification ponctuelle.

Le présent rapport synthétise l'ensemble du projet à travers l'architecture retenue, le protocole expérimental et les résultats consolidés, en mettant l'accent sur la conformité fonctionnelle, la performance et sa sensibilité à l'accélération matérielle selon la plateforme, la stabilité statistique et la reproductibilité.

Problématique

Le chiffrement symétrique est largement mobilisé pour la protection des données, mais son évaluation demeure souvent fragmentée entre robustesse théorique, performance brute et contraintes matérielles. En pratique, un algorithme peut présenter de bonnes propriétés cryptographiques tout en perdant en pertinence opérationnelle selon la plateforme ciblée. L'écart entre une architecture x86 avec accélération matérielle et une architecture ARM où l'accélération AES n'est pas explicitement exploitée dans le protocole expérimental peut, à lui seul, modifier de façon substantielle les conclusions de performance.

La question centrale du projet est donc la suivante. Comment comparer, de manière rigoureuse et reproductible, plusieurs algorithmes de chiffrement symétrique dans un cadre

expérimental unique, sans introduire de biais liés aux différences d'implémentation, de mode d'opération ou d'environnement d'exécution.

Cette problématique se décline en quatre enjeux concrets. Le premier enjeu consiste à établir une base de comparaison uniforme pour AES, DES, 3DES, Twofish et ChaCha20 sur x86 et ARM. Le deuxième enjeu porte sur la vérification de la conformité fonctionnelle de chaque implémentation au moyen de KAT. Le troisième enjeu concerne la mesure des performances avec un niveau de crédibilité statistique explicite, via l'IC95. Le quatrième enjeu vise l'évaluation des propriétés de diffusion, notamment l'effet d'avalanche, et l'interprétation de leur cohérence au regard de la littérature.

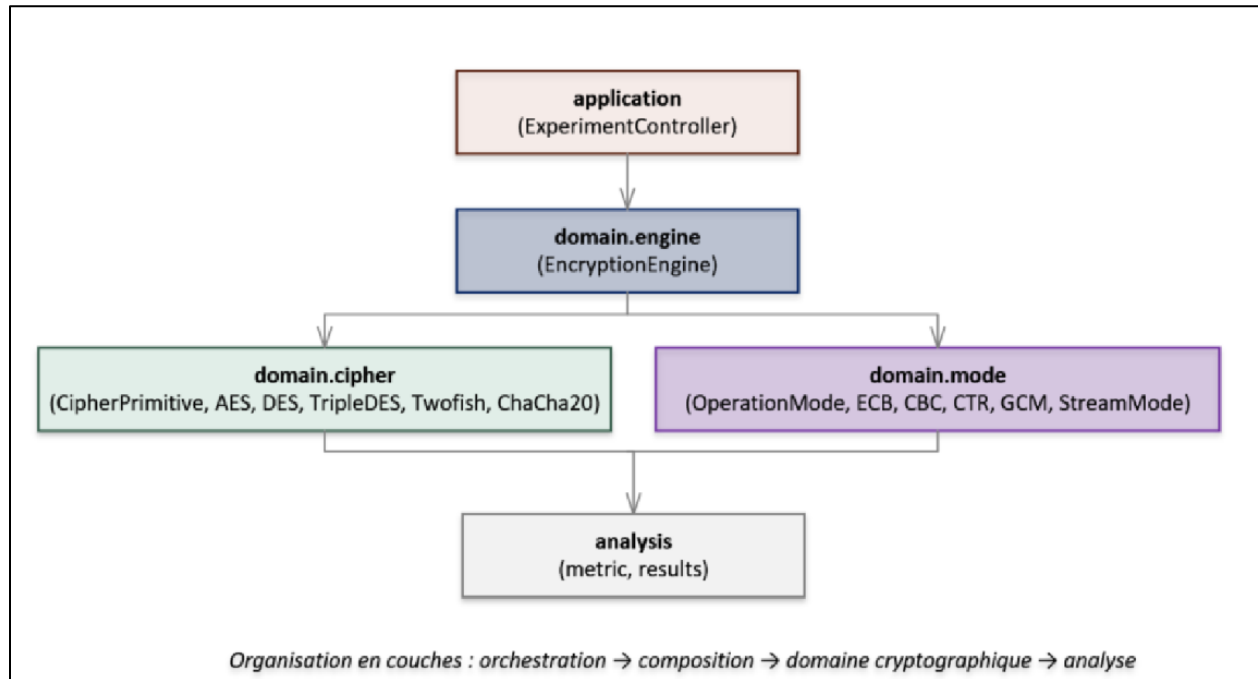
En résumé, la problématique ne se limite pas à identifier l'algorithme le plus rapide. Elle vise à produire une comparaison fiable, traçable et utile à la décision, en tenant compte des conditions matérielles réelles d'utilisation.

Architecture du système

L'architecture adopte une conception orientée objet, modulaire et en couches, fondée sur une séparation stricte de trois responsabilités, la primitive cryptographique, le mode d'opération et le moteur d'orchestration qui combine ces deux éléments. Ce découplage applique les principes de faible couplage et de forte cohésion, ce qui permet d'ajouter un nouvel algorithme ou un nouveau mode sans modifier les autres couches, tout en maintenant un cadre expérimental uniforme pour la comparaison inter-algorithmes.

Cette organisation se traduit concrètement par une structure en couches distinctes, illustrée à la figure suivante selon un flux d'orchestration, de composition et de domaine cryptographique. La couche application orchestre les campagnes expérimentales sans connaître les détails internes des primitives, la couche domaine encapsule séparément les algorithmes de chiffrement et les modes d'opération, tandis que les métriques produites par cette orchestration alimentent directement les résultats présentés dans les sections suivantes du rapport.

Figure 1 – Diagramme de paquets du système (vue en couches)



Le composant CipherPrimitive définit l'interface abstraite commune aux primitives cryptographiques du système. Chaque implémentation concrète, notamment AES, DES, 3DES, Twofish et ChaCha20, hérite de cette interface et implémente encrypt_block, decrypt_block, ainsi que les propriétés block_size et key_size. Cette abstraction isole la transformation cryptographique élémentaire et exclut toute logique de chaînage, de vecteur d'initialisation ou de nonce.

Le composant OperationMode encapsule une primitive et applique une stratégie de traitement des blocs afin de gérer des messages de longueur arbitraire. Les modes ECB (Electronic Codebook), CBC (Cipher Block Chaining), CTR (Counter Mode) et GCM suivent cette interface. En parallèle, StreamMode joue le rôle de passerelle pour les chiffrements de flux comme ChaCha20, qui gèrent intrinsèquement leur nonce et ne reposent pas sur un chaînage bloc-à-bloc.

Le composant EncryptionEngine combine une primitive et un mode déjà initialisés pour former un service de chiffrement complet. Il expose une interface unifiée, centrée sur les

méthodes encrypt et decrypt, utilisée de manière identique par la couche applicative, indépendamment de l'algorithme et du mode sous-jacents.

Au-dessus de la couche domaine, ExperimentController représente la couche application. Ce composant orchestre les campagnes de mesure, incluant les performances, l'effet d'avalanche et l'effet d'avalanche de clé, sans dépendre des détails internes des primitives et des modes. Cette séparation améliore la traçabilité des expériences, limite les biais d'implémentation et renforce la reproductibilité des résultats.

Cette architecture a été implémentée en Python, un choix qui ne repose pas sur la performance brute du langage lui-même, mais sur sa capacité à assurer une intégration homogène des primitives cryptographiques et une automatisation cohérente des campagnes expérimentales sur les deux plateformes. Cette neutralité tient notamment au fait que PyCryptodome délègue les opérations cryptographiques critiques à des routines C compilées et optimisées, identiques sur x86 et sur ARM, de sorte que les écarts de performance mesurés reflètent les contraintes matérielles des plateformes plutôt qu'une différence d'implémentation logicielle. Python sert ainsi de couche d'orchestration reproductible autour d'un même noyau natif, ce qui est précisément l'objet de la comparaison.

Protocole experimental

Le protocole expérimental vise à mesurer, de manière comparable, les performances et les propriétés de diffusion des cinq algorithmes retenus, dans des conditions contrôlées et reproductibles sur les deux plateformes matérielles.

Matrice expérimentale

L'expérimentation couvre 31 combinaisons distinctes d'algorithme, de mode d'opération et de taille de clé. AES est évalué en ECB, CBC, CTR et GCM avec des clés de 128, 192 et 256 bits. DES est évalué en ECB, CBC et CTR avec une clé nominale de 64 bits, dont 56 bits effectifs de sécurité. 3DES est évalué en ECB, CBC et CTR avec des clés de 128 et 192 bits.

Twofish est évalué en ECB, CBC et CTR avec des clés de 128, 192 et 256 bits. ChaCha20 est évalué uniquement en StreamMode avec une clé de 256 bits, puisqu'il s'agit d'un chiffrement de flux qui ne s'intègre pas aux modes de chaînage par blocs.

Pour chaque combinaison, cinq tailles de message sont utilisées, soit 64, 256, 1024, 4096 et 16384 octets. Ce choix permet d'observer le comportement des algorithmes à la fois sur de petits messages et sur des volumes plus représentatifs d'un usage réel.

Mesure du débit

Pour chaque configuration définie par l'algorithme, le mode, la taille de clé et la taille de message, le script génère une clé aléatoire puis un texte en clair aléatoire de longueur fixée. Ce même texte en clair est ensuite réutilisé pour 100 répétitions de chiffrement et 100 répétitions de déchiffrement afin d'obtenir des moyennes comparables. Le chronométrage encadre uniquement l'appel cryptographique, ce qui limite l'effet des surcoûts de l'interpréteur et de l'allocation mémoire. Le déchiffrement est mesuré séparément à partir du dernier texte chiffré produit. Le débit en mégaoctets par seconde est ensuite calculé à partir du temps moyen d'exécution.

Intervalle de confiance à 95%

Pour chaque configuration, cent répétitions sont exécutées séparément pour le chiffrement et pour le déchiffrement, puis les temps observés sont agrégés en moyenne avec estimation de la dispersion. L'IC95 est ensuite dérivé de cette variabilité mesurée sur les répétitions afin d'encadrer l'incertitude autour du débit estimé. Concrètement, il est calculé à partir de l'écart-type des répétitions, avec une approximation normale pour les séries de taille suffisante, ce qui correspond ici à 100 répétitions. Il indique la plage dans laquelle la performance réelle a une forte probabilité de se situer, plutôt qu'une valeur ponctuelle isolée. Un intervalle étroit traduit une mesure stable et reproductible, alors qu'un intervalle large signale une sensibilité élevée au bruit expérimental. Ces intervalles servent directement de base aux seuils de qualité de l'audit IC95, afin de distinguer une variation de performance structurelle d'une fluctuation de mesure.

Effet d'avalanche

En complément du débit, deux mesures de diffusion sont évaluées directement au niveau de la primitive cryptographique, donc sans influence du mode d'opération. La première mesure perturbe le texte en clair en inversant exactement un bit, puis quantifie la fraction de bits qui changent dans le texte chiffré obtenu. La seconde reprend le même protocole en inversant un bit de la clé, afin d'estimer la sensibilité de la sortie à une variation minimale du secret. Chaque mesure est répétée sur un ensemble d'essais indépendants, puis agrégée par moyenne pour réduire le bruit d'échantillonnage. La valeur cible attendue reste proche de 50 %, ce qui correspond à une diffusion équilibrée conforme au critère de diffusion stricte discuté par Stallings (2017) et au cadre confusion-diffusion de Katz et Lindell (2020).

Reproductibilité entre plateformes

Le protocole complet est exécuté indépendamment sur x86 puis sur Raspberry Pi, avec une matrice expérimentale strictement identique entre les deux environnements, donc les mêmes combinaisons algorithme mode clé, les mêmes tailles de message et le même volume de répétitions par mesure. Les clés et les messages sont générés en octets bruts via des appels à `os.urandom`, sans conversion de chaînes de caractères dépendante de l'encodage du système d'exploitation. Cette méthode réduit les écarts potentiels Linux Windows liés au traitement textuel et maintient une base binaire homogène entre plateformes. Chaque campagne produit un export CSV daté, conservé sans post-traitement destructif, afin de préserver la traçabilité des observations au niveau brut. Cette conservation permet une comparaison interplateforme directement à partir des mesures originales, puis une régénération reproductible des agrégats et des graphiques à partir des mêmes sources de données.

Validation fonctionnelle (KAT)

Avant toute mesure de performance, chaque implémentation cryptographique est validée fonctionnellement au moyen de tests à réponses connues, les KAT (Known Answer Tests).

Cette validation vérifie que chaque primitive produit exactement les textes chiffrés attendus pour des vecteurs d'entrée normalisés, indépendamment de toute considération de vitesse.

Sources normatives

Les vecteurs de validation utilisés dans ce projet proviennent de références normatives reconnues. Le tableau suivant associe chaque algorithme et chaque mode à sa source de validation, afin de rendre la méthode explicite et vérifiable.

Tableau 1 — Références normatives des tests KAT

Élément validé	Référence normative	Portée de validation
AES	FIPS 197	Primitive AES
DES	FIPS 46-3 et NIST SP 800-17	Primitive DES et procédures de test
3DES	NIST SP 800-67	Cas à deux clés et à trois clés
Modes ECB CBC CTR	NIST SP 800-38A	Modes de chiffrement par blocs
GCM	NIST SP 800-38D	Chiffrement authentifié et tag d'intégrité
ChaCha20	RFC 8439	Chiffrement de flux
Twofish	Vecteurs officiels Schneier et al.	Familles ECB_VK, ECB_VT, ECB_TBL, avec un vecteur canonique par famille (chiffrement et déchiffrement) dans le profil principal, et l'ensemble des vecteurs disponibles dans le profil complet

Ce tableau confirme que chaque implémentation est validée contre une référence adaptée à sa nature cryptographique. Cette traçabilité normative renforce la crédibilité des résultats de performance présentés dans la suite du rapport.

Deux profils d'exécution

Le script de validation propose deux profils d'exécution. Les suites AES, DES, 3DES, Modes, GCM et ChaCha20 restent identiques dans les deux profils, et seule la couverture Twofish varie entre profil principal et profil complet. Le profil principal valide 60 assertions sur 60 et

sert de vérification rapide en développement. Le profil complet valide 2268 assertions sur 2268 et alimente le dossier de preuves officiel.

Intégrité des vecteurs externes

Les vecteurs Twofish externes sont distribués avec leurs fichiers d'empreinte .sha256. Avant toute exécution des tests KAT, le script recalcule l'empreinte de chaque fichier de vecteurs et la compare à la valeur attendue. Cette vérification garantit l'intégrité des jeux de test et permet de détecter toute altération, corruption ou substitution de fichier entre deux exécutions ou entre deux environnements.

Bilan de l'exécution

Le profil principal valide de manière constante 60 assertions sur 60 pour l'ensemble des sept suites, AES, DES, 3DES, Modes, GCM, ChaCha20 et Twofish. Le profil complet valide de la même façon 2268 assertions sur 2268. Ces totaux sont fixes, puisqu'ils correspondent au nombre de vecteurs de test normalisés couverts par chaque profil, plutôt qu'à un résultat susceptible de varier d'une exécution à l'autre, ce que confirme leur reproduction identique lors des exécutions automatiques du pipeline d'intégration continue. Aucun échec n'est observé dans les deux profils, ce qui confirme la conformité fonctionnelle de toutes les implémentations avant leur intégration au protocole expérimental de performance.

Afin de ne pas dépendre uniquement d'une exécution manuelle ponctuelle, cette suite de validation est intégrée à une chaîne d'intégration continue (CI, Continuous Integration) exécutée automatiquement à chaque changement du dépôt, au moyen de deux flux de travail GitHub Actions. Le premier relance l'ensemble des tests avec pytest à chaque push et chaque pull request, publie les rapports de couverture de code, puis vérifie l'absence de vulnérabilités connues dans les dépendances à l'aide de pip-audit. Le second est dédié à l'analyse de sécurité du code et combine une analyse statique par CodeQL et Bandit, ainsi qu'un audit distinct des dépendances. Les outils de cette chaîne de vérification sont isolés dans un fichier de dépendances de développement distinct, afin de ne pas alourdir l'environnement d'exécution du système expérimental. Un mécanisme Dependabot

complète ce dispositif en proposant automatiquement une mise à jour hebdomadaire des dépendances.

Cette automatisation garantit qu'aucune régression ne peut être introduite silencieusement dans les implémentations cryptographiques et prolonge la logique méthodologique déjà établie dans cette section, où la vérification de conformité précède systématiquement toute mesure de performance. Au-delà de cet intérêt méthodologique immédiat, cette intégration traduit une volonté d'aligner le projet sur les pratiques actuelles du développement logiciel professionnel, dans lesquelles validation et sécurité sont traitées comme des processus continus intégrés au cycle de développement (DevSecOps), plutôt que comme des étapes isolées de fin de parcours.

Résultats et analyse

Cette section présente une synthèse des principaux résultats expérimentaux obtenus pour les cinq algorithmes sur les deux plateformes. L'analyse met l'accent sur trois axes, le débit de chiffrement, la stabilité statistique des mesures et l'effet d'avalanche. Ces axes correspondent directement aux objectifs de performance et de diffusion définis en introduction. Ils sont traités dans cet ordre afin de construire l'argumentation de façon progressive, d'abord la performance brute observée, ensuite la fiabilité statistique des mesures, puis une propriété cryptographique indépendante du débit. Chaque résultat est mis en relation, lorsque pertinent, avec les sources normatives et la littérature mobilisées dans le rapport.

Débit de chiffrement selon la plateforme

Le tableau suivant présente les débits maximaux observés en chiffrement, exprimés en mégaoctets par seconde, en retenant pour chaque algorithme la taille de message, parmi celles testées, ayant produit le débit le plus élevé. Les valeurs reprises ici proviennent de la même campagne finale consolidée que le reste de la section. Contrairement au premier graphique, qui fixe une taille de message unique de 4096 octets afin de comparer les

algorithmes dans des conditions strictement identiques, ce tableau vise à capturer la performance de pointe atteignable par chaque algorithme.

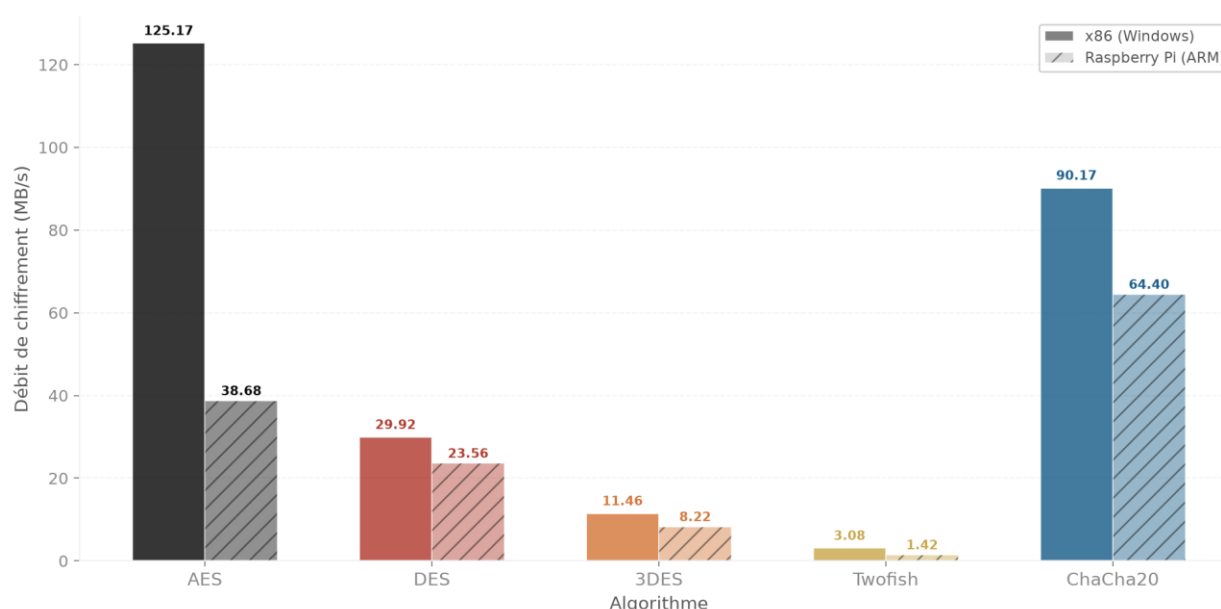
Tableau 2 — Débit maximal de chiffrement par algorithme, x86 versus ARM

Algorithme	x86 (MB/s)	Pi ARM (MB/s)	Ratio x86/Pi
AES	373,74	55,75	6,70×
ChaCha20	144,55	89,35	1,62×
DES	38,78	31,85	1,22×
Twofish	3,14	1,44	2,18×
3DES	13,80	11,16	1,24×

Le deuxième tableau met en évidence une hiérarchie nette des débits maximaux. AES atteint le niveau le plus élevé sur les deux plateformes et son ratio x86/Pi monte à 6,70. ChaCha20 suit à 1,62. DES et 3DES restent proches de 1,22 et 1,24. Twofish demeure le plus lent. Cette distribution s'explique principalement par la présence d'AES-NI sur x86, absente sur Raspberry Pi, tandis que les autres algorithmes reposent davantage sur des chemins logiciels sans accélération dédiée. Cette lecture rejoint l'analyse de Stallings (2017) sur l'influence conjointe de l'algorithme et de l'architecture d'exécution. En conséquence, un débit maximal élevé reflète ici autant le design cryptographique que l'adéquation entre l'algorithme et les capacités matérielles de la plateforme cible. Twofish se distingue en outre par une croissance quasi nulle de son débit à mesure que la taille du message augmente, contrairement aux trois autres chiffrements par blocs, ce qui suggère un goulot d'étranglement dominé par le coût de la primitive elle-même plutôt que par les coûts fixes d'appel.

Afin d'isoler l'effet de la plateforme, la comparaison suivante fixe la taille du message à 4096 octets. Le graphique présente, pour chaque algorithme, une paire de barres opposant la plateforme x86 sous Windows au Raspberry Pi sous architecture ARM, en mégaoctets par seconde. Comme pour le deuxième tableau, la configuration de référence correspond au mode ECB pour les chiffrements par blocs et au mode natif Stream pour ChaCha20.

Graphique 1 — Débit par algorithme, x86 versus ARM à 4096 octets



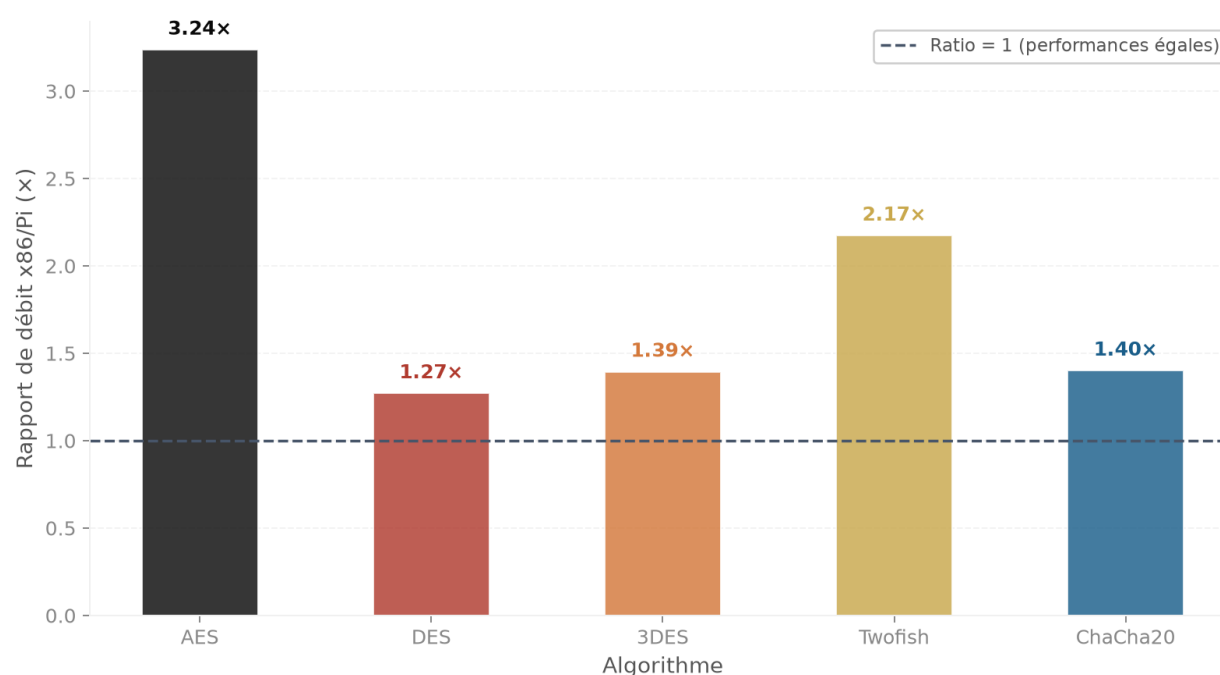
Cette figure confirme cette tendance dans un cadre à taille contrôlée. À 4096 octets, AES reste en tête avec un ratio x86/Pi de 3,24. Twofish suit à 2,17, puis ChaCha20, 3DES et DES autour de 1,40, 1,39 et 1,27. L'accélération matérielle demeure le facteur principal pour AES. Toutefois, l'écart plus faible qu'au débit maximal montre aussi que l'amortissement des coûts fixes dépend de la taille du message, ce qui réduit partiellement l'avantage observé en condition de pointe. Sur le plan méthodologique, la comparaison inter-plateforme doit donc toujours être lue à taille de message explicite, car un classement établi à débit maximal ne se transpose pas automatiquement à une charge représentative.

Ces deux premières lectures, débit de pointe au deuxième tableau et débit à taille fixe au premier graphique, remplissent des rôles méthodologiques distincts qui se complètent plutôt qu'ils ne se répètent. Le débit maximal renseigne sur la capacité de traitement la plus élevée qu'un algorithme peut atteindre dans les conditions les plus favorables du protocole, une information utile pour dimensionner un système autour de sa charge de pointe. Le débit à 4096 octets, à l'inverse, reflète une taille de message représentative d'un usage courant, plus proche de ce qu'on rencontre concrètement dans des échanges réseau typiques ou des blocs de stockage usuels, plutôt qu'un scénario optimisé pour maximiser le débit. Cette distinction explique en partie pourquoi le ratio d'AES se contracte notablement entre les

deux mesures, le gain apporté par les instructions AES-NI dépend d'un coût fixe amorti sur chaque appel, notamment la préparation du calendrier de clés, un coût qui pèse proportionnellement moins lorsque le message traité est volumineux. Cette nuance justifie de ne pas se fier à une seule de ces deux lectures pour orienter un choix d'algorithme, la performance de pointe et la performance représentative pouvant classer les mêmes algorithmes différemment selon le contexte d'utilisation réel visé.

Les deux lectures précédentes mettent en évidence des écarts qui varient selon l'algorithme. Pour quantifier cet effet, le graphique suivant exprime le rapport entre le débit x86 et le débit Raspberry Pi, dans la même configuration que le premier graphique, à 4096 octets. Un ratio proche de 1 traduit une portabilité de performance élevée, alors qu'un ratio plus grand indique une dépendance accrue au matériel x86.

Graphique 2 — Ratio d'accélération x86 sur ARM par algorithme

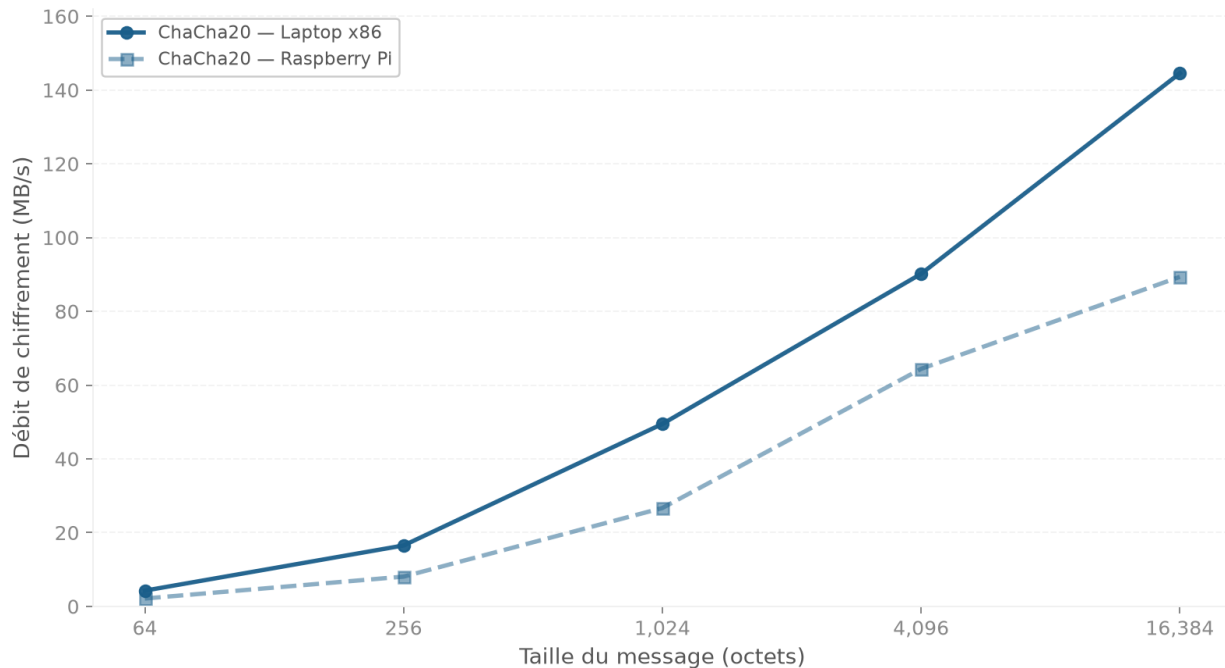


Cette figure confirme que la dépendance à la plateforme diffère selon l'algorithme. AES présente le ratio x86/Pi le plus élevé à 3,24, suivi de Twofish à 2,17, puis de ChaCha20, 3DES et DES à 1,40, 1,39 et 1,27. Ce profil combine l'effet d'AES-NI, l'absence d'instructions dédiées pour DES et 3DES, et un coût logiciel de Twofish plus sensible aux contraintes de

cache et de fréquence sur ARM. Il en résulte que la portabilité ne peut pas être inférée d'un seul algorithme. Les gains dus au matériel doivent donc être distingués de ceux liés à l'implémentation.

La lecture de ce ratio soulève une question naturelle, le comportement observé à 4096 octets reste-t-il stable lorsque la taille des messages varie. Le cas de ChaCha20 est particulièrement pertinent pour cette vérification, car il constitue le seul chiffrement de flux de l'échantillon et présente un ratio intermédiaire d'environ 1,40. Le troisième graphique suit donc son débit sur les deux plateformes de 64 à 16 384 octets.

Graphique 3 — Débit en fonction de la taille du message, ChaCha20 x86 versus ARM



Le troisième graphique montre que l'écart x86 versus Raspberry Pi pour ChaCha20 varie avec la taille du message et ne suit pas une trajectoire monotone. Cette variation est cohérente avec l'équilibre entre coûts fixes et coûts proportionnels. Les petites tailles amplifient les surcoûts d'appel, alors que les grandes tailles les amortissent davantage, avant que la hiérarchie mémoire et l'ordonnancement ne reprennent du poids. Cette observation indique qu'un point unique de mesure ne suffit pas à caractériser la portabilité

de ChaCha20. Une lecture multi-tailles, appuyée par le tableau numérique suivant, est donc nécessaire.

Tableau 3 — Débit de ChaCha20 selon la taille du message

Taille du message (octets)	x86 (MB/s)	Pi ARM (MB/s)	Ratio x86/Pi
64	4,27	2,13	2,00×
256	16,52	8,07	2,05×
1024	49,46	26,68	1,85×
4096	90,17	64,40	1,40×
16384	144,55	89,35	1,62×

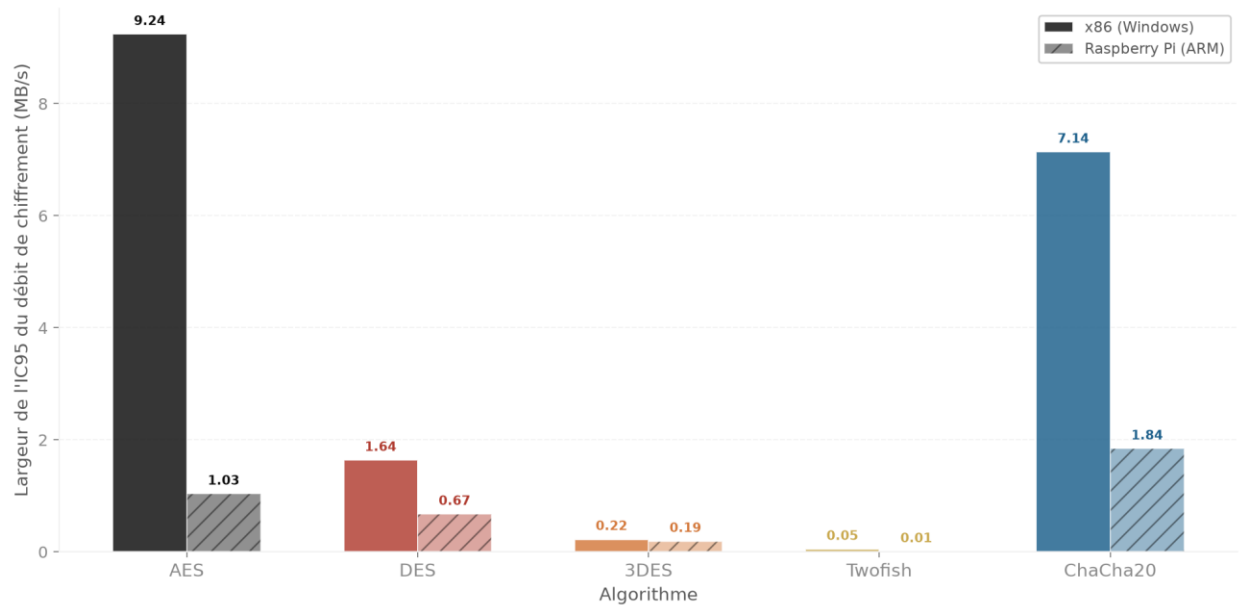
Le troisième tableau confirme ce comportement de manière chiffrée. Le ratio x86/Pi de ChaCha20 oscille entre 1,40 et 2,05. À 16384 octets, ChaCha20 sur Raspberry Pi atteint 89,35 MB/s, soit un débit supérieur à celui mesuré pour DES sur x86 au même point de mesure (38,78 MB/s en mode ECB). Ce résultat est cohérent avec le design ARX de ChaCha20 et son efficacité logicielle sur processeurs généralistes telle que décrite dans la RFC 8439 (Nir et Langley, 2018), tandis que le coût plus élevé de transformations bit à bit dans DES est cohérent avec la littérature de référence sur les chiffrements par blocs classiques (Stallings, 2017). Ainsi, ChaCha20 présente une robustesse inter-plateforme solide pour des charges réelles, tandis que Twofish demeure le cas le plus pénalisé en performance relative sans accélération matérielle dédiée.

Stabilité statistique des mesures

Après l'analyse des niveaux de débit, il est nécessaire de qualifier la robustesse statistique des mesures. Cette stabilité est évaluée au moyen de l'IC95 sur l'ensemble des campagnes x86 et Raspberry Pi. Sur 310 mesures agrégées, 297 respectent un seuil de largeur relative d'au plus 10 %, soit 95,81 % des configurations. Le taux atteint 97,31 % pour les messages d'au moins 1024 octets, avec 181 mesures conformes sur 186, contre 93,55 % pour les tailles 64 et 256 octets, avec 116 mesures conformes sur 124. Cette distribution confirme

que le bruit relatif se concentre surtout sur les petites tailles, où les durées d'exécution sont très brèves malgré 100 répétitions par configuration.

Graphique 4 — Stabilité du débit IC95, x86 versus ARM à 4096 octets

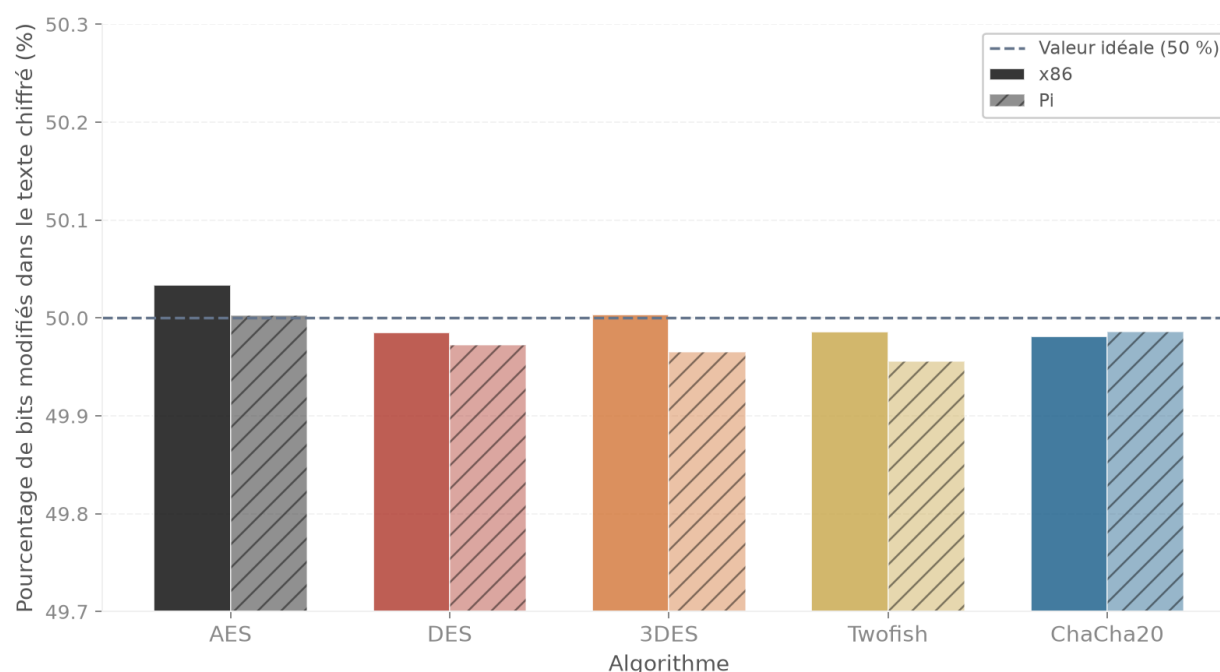


Cette figure confirme une stabilité globale élevée des mesures. La figure correspond à la configuration 4096 octets avec 100 répétitions par mesure, cohérente avec le protocole IC95 décrit plus haut. À 4096 octets, AES sur x86 atteint une largeur absolue de 9,24 MB/s, ce qui correspond à une largeur relative d'environ 7,38 % au regard du débit mesuré sur ce même point. Les autres largeurs absolues sur x86 sont de 7,14 MB/s pour ChaCha20, 1,64 MB/s pour DES, 0,22 MB/s pour 3DES et 0,05 MB/s pour Twofish, tandis que du côté Raspberry Pi on observe 1,03 MB/s pour AES, 1,84 MB/s pour ChaCha20, 0,67 MB/s pour DES, 0,19 MB/s pour 3DES et 0,01 MB/s pour Twofish. Cette situation s'explique par le fait que les algorithmes les plus rapides convertissent de petites fluctuations temporelles en variations MB/s plus visibles, avec une sensibilité possible aux variations de fréquence et aux tâches d'arrière-plan sur l'hôte x86. Les écarts moyens restent donc interprétables, tout en justifiant une prudence statistique explicite pour les comparaisons fines.

Effet d'avalanche

Au-delà du débit et de sa stabilité, l'analyse doit aussi couvrir la qualité de diffusion cryptographique. L'effet d'avalanche mesure la proportion de bits du texte chiffré modifiés lorsqu'un seul bit du texte clair est inversé. Un chiffrement robuste doit idéalement produire une variation proche de la moitié des bits de sortie pour une perturbation minimale en entrée. Le graphique suivant présente le score moyen obtenu pour chaque algorithme sur x86 et sur Raspberry Pi.

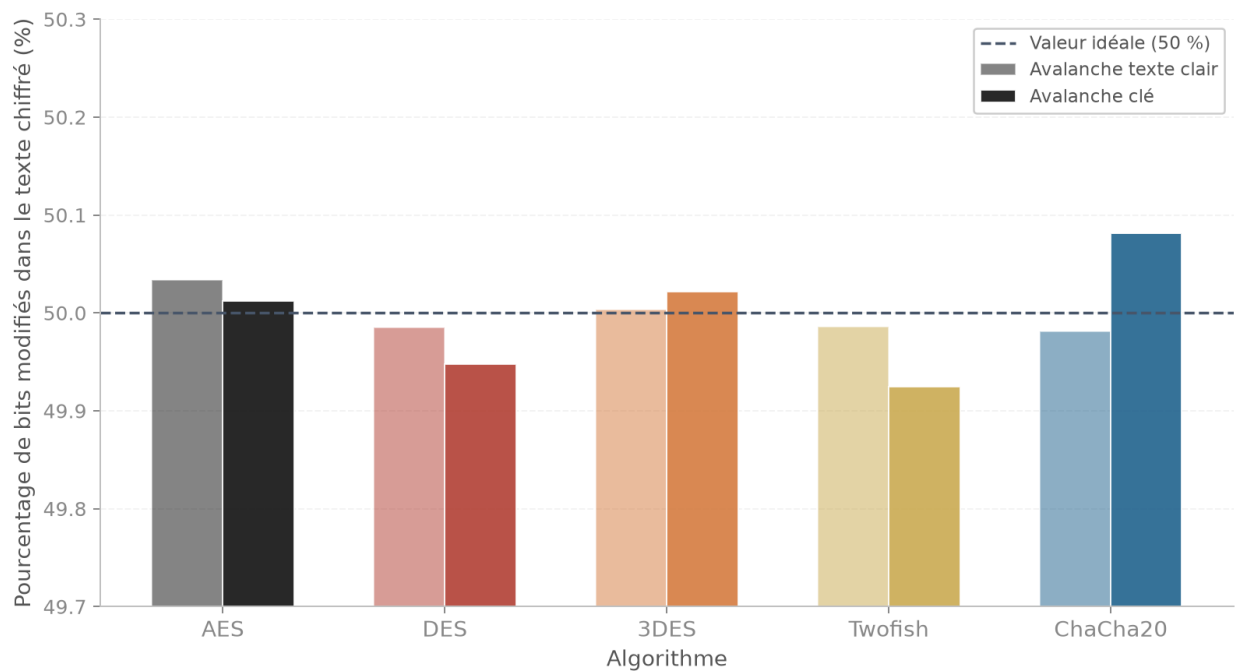
Graphique 5 — Score d'avalanche par algorithme, x86 versus ARM



Le cinquième graphique montre une diffusion correcte et homogène, les cinq algorithmes restant très proches de l'idéal de 50 % sur x86 comme sur Raspberry Pi, avec un écart interplateforme maximal de 0,038 point de pourcentage. L'axe vertical est volontairement resserré autour de 50 % afin de rendre visibles les écarts fins entre algorithmes et plateformes. Ce résultat confirme empiriquement la cible théorique déjà justifiée dans le protocole expérimental. La qualité de diffusion peut donc être considérée comme stable d'une plateforme à l'autre dans cette campagne, ce qui évite de confondre variation de performance et robustesse cryptographique.

L'étape suivante consiste à distinguer les deux sources de diffusion, une perturbation du texte clair et une perturbation de la clé. Cette comparaison permet de vérifier si chaque algorithme conserve une réponse homogène entre ces deux mécanismes. Le graphique suivant confronte, pour la plateforme x86, le score d'avalanche lié au texte clair et celui lié à la clé.

Graphique 6 — Avalanche du texte en clair versus avalanche de la clé

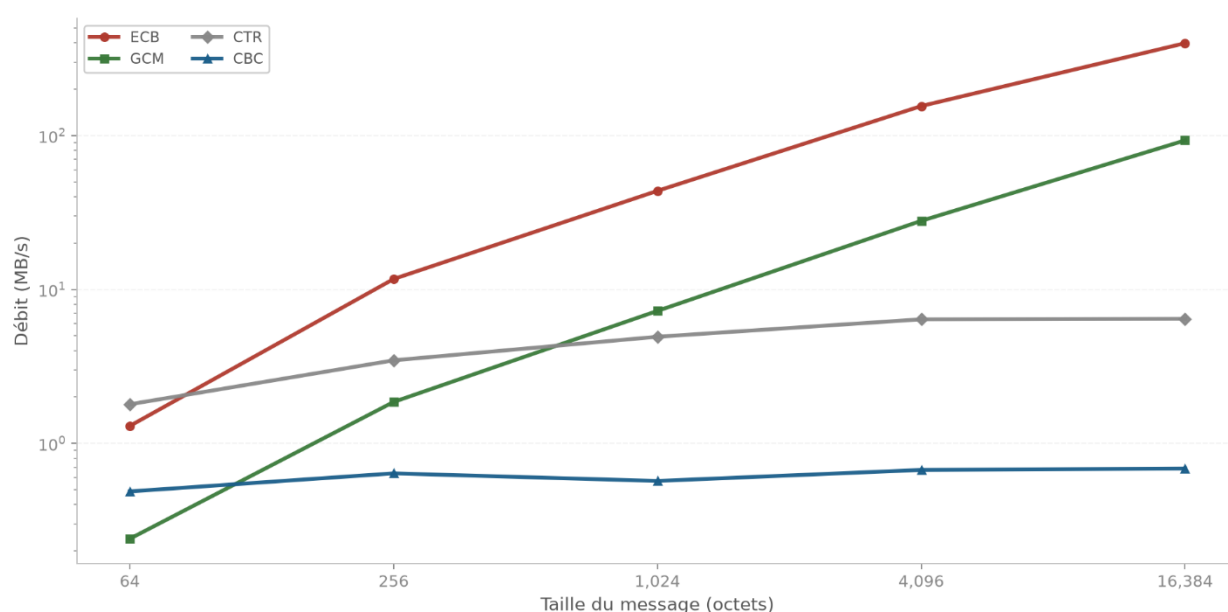


La sixième figure confirme la cohérence interne des mesures d'avalanche. Comme dans la figure précédente, l'échelle est centrée autour de 50 % pour privilégier la lisibilité des écarts faibles entre les deux types de perturbation. Pour AES, DES, 3DES et Twofish, l'écart entre avalanche du texte clair et avalanche de la clé reste inférieur à 0,1 point de pourcentage. ChaCha20 demeure très proche avec un écart d'environ 0,10 point. Cette proximité s'explique par le fait que les deux perturbations testent la même capacité de diffusion globale du chiffrement. Les scores retenus peuvent ainsi être interprétés comme homogènes entre les deux mécanismes de test, ce qui renforce la validité comparative de l'analyse d'avalanche.

Modes de chiffrement

Après la comparaison inter-algorithmes, l'analyse se recentre sur l'effet du mode d'opération pour un même algorithme. Ce changement d'échelle est important, car le mode influence simultanément le débit et le niveau de sécurité opérationnelle. Le graphique suivant présente le débit d'AES-128 en ECB, CBC, CTR et GCM sur la plateforme x86 pour différentes tailles de message.

Graphique 7 — Débit et niveau de sécurité selon le mode d'opération AES

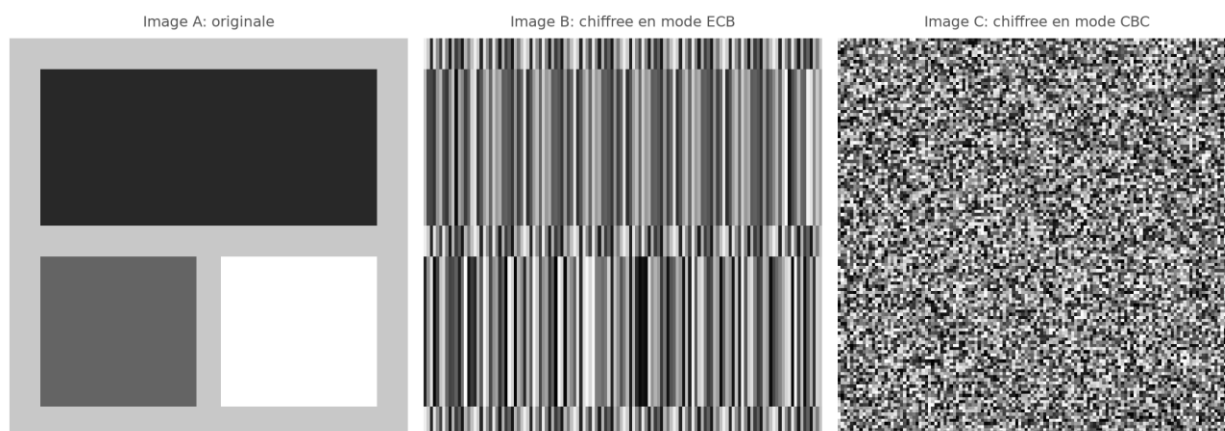


Le septième graphique révèle un comportement plus nuancé qu'un simple classement fixe. À petite taille de message, CTR devance légèrement ECB, tandis que GCM demeure le plus lent des quatre modes. Cet ordre s'inverse ensuite, GCM dépassant CTR autour de 1024 octets puis se rapprochant progressivement du débit d'ECB, alors que CTR plafonne autour de 6 MB/s au-delà de cette taille. CBC, pour sa part, demeure pratiquement plat sur l'ensemble des tailles testées, entre 0,5 et 0,7 MB/s, ce qui reste cohérent avec le chaînage séquentiel de ce mode. Bien qu'ECB et CTR permettent tous deux, en principe, le calcul parallèle de plusieurs blocs selon NIST SP 800-38A, seul ECB manifeste ici une croissance linéaire soutenue. Ce contraste s'explique par l'implémentation retenue dans ce projet, où CTR construit les blocs compteur et applique le XOR dans le code Python, alors que GCM

délègue le chiffrement authentifié à l'appel natif AES-GCM de PyCryptodome. Cet écart d'implémentation, plutôt qu'une limite du mode CTR lui-même, illustre une tension plus générale entre la flexibilité qu'offre une implémentation Python modulaire et le débit atteignable par un appel natif groupé, une tension qui n'affecte pas de la même façon les quatre modes étudiés selon qu'ils délèguent ou non l'essentiel du traitement à la bibliothèque cryptographique sous-jacente. En pratique, le choix d'un mode ne peut donc pas être guidé par le seul débit et doit intégrer simultanément le niveau de sécurité attendu dans le scénario visé.

La supériorité d'ECB en débit ne suffit toutefois pas à établir sa pertinence en pratique. La démonstration suivante explicite la limite structurelle du mode en comparant visuellement le chiffrement d'une même image en ECB et en CBC.

Graphique 8 — Fuite visuelle des motifs en mode ECB

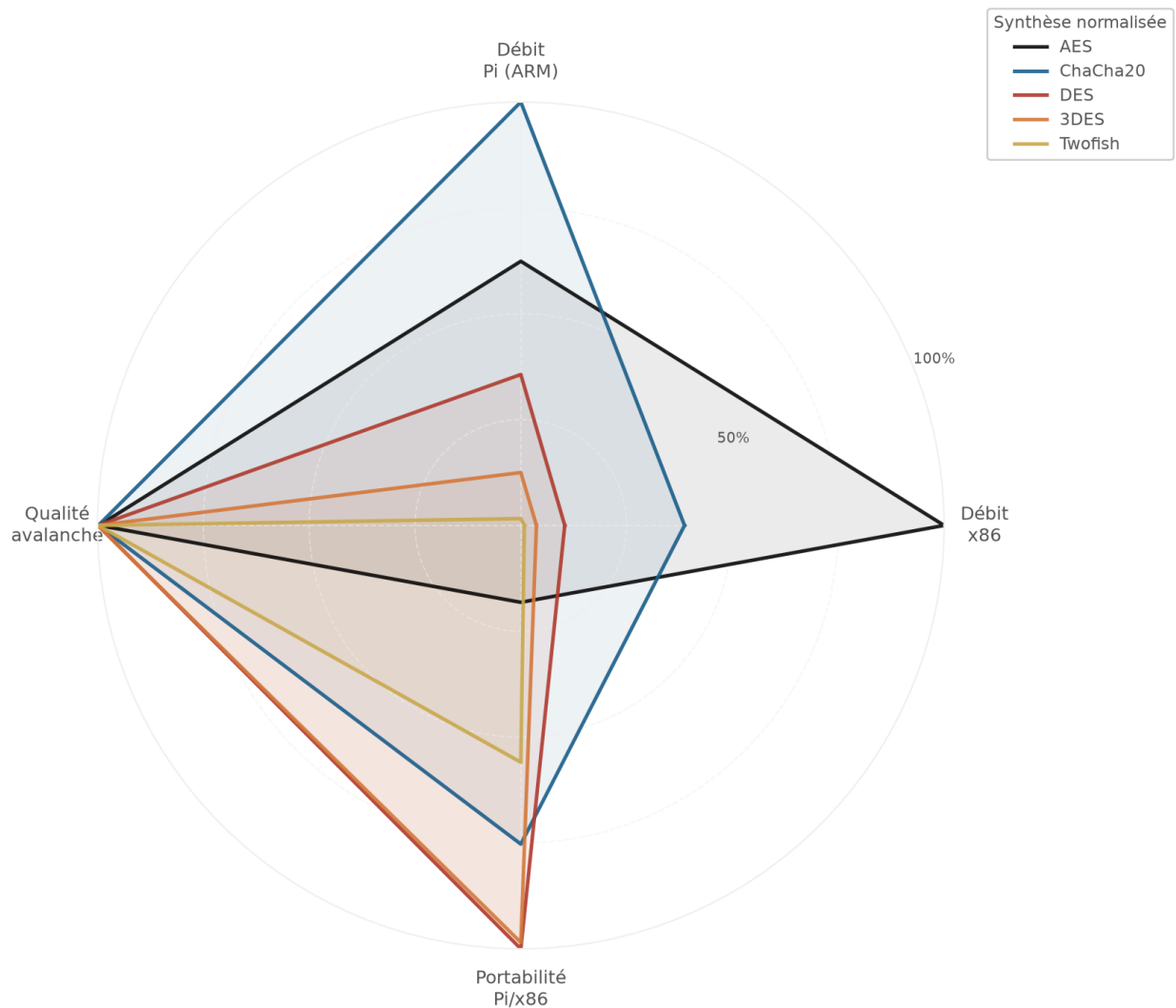


Le huitième graphique fournit une démonstration visuelle directe, l'image chiffrée en ECB conserve des motifs répétitifs alors que la version CBC apparaît comme un bruit homogène sans structure exploitable. Cette différence provient du fonctionnement déterministe d'ECB sur des blocs identiques, alors que le chaînage de CBC casse ces répétitions d'un bloc à l'autre. Par conséquent, ECB doit être écarté des usages réels malgré son avantage de débit, en cohérence avec NIST SP 800-38A qui recommande des modes non déterministes fondés sur un vecteur d'initialisation (IV) ou un nonce unique pour les données structurées.

Vue synthétique multicritère

Les analyses précédentes fournissent des lectures partielles complémentaires. Pour consolider l'interprétation, le graphique suivant réunit les résultats dans une représentation globale. Le diagramme radar combine quatre axes normalisés, le débit sur x86, le débit sur Raspberry Pi, la qualité de l'effet d'avalanche et la portabilité définie par le rapport Pi sur x86.

Graphique 9 — Radar de profil par algorithme, débit, portabilité et avalanche

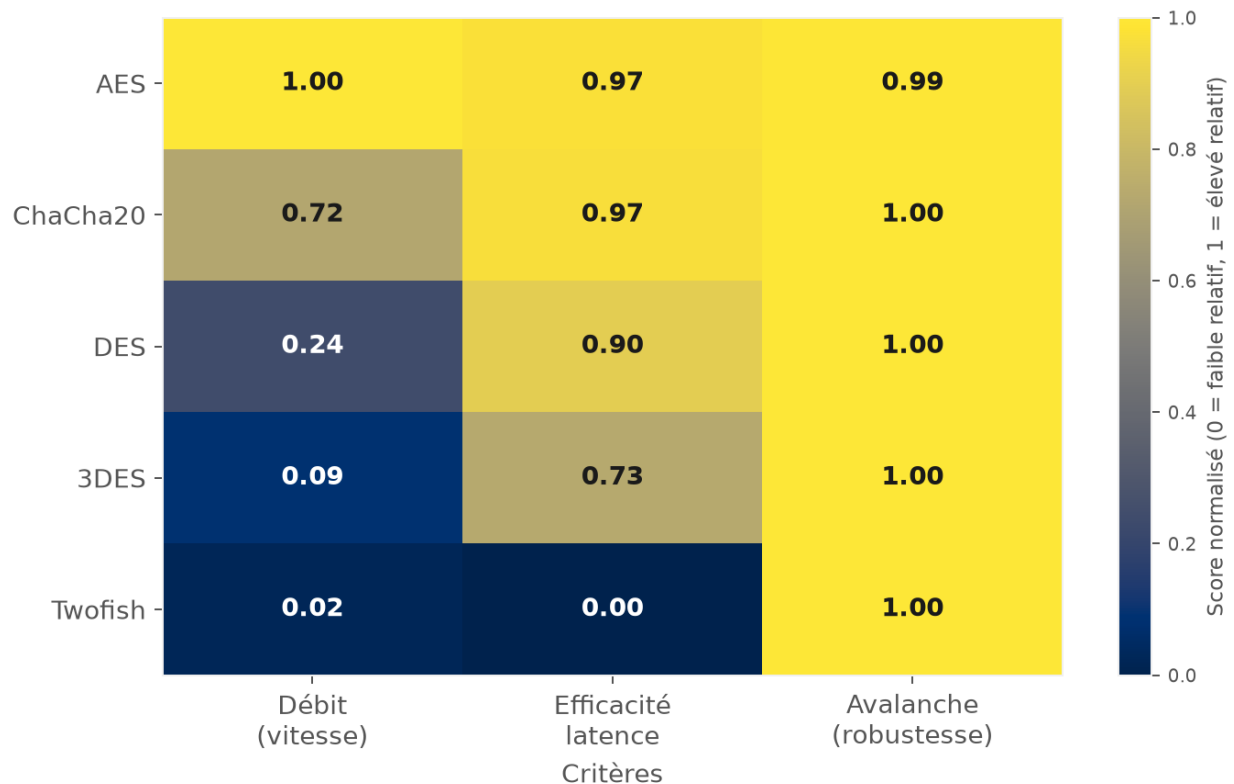


Le neuvième graphique met en évidence un compromis multicritère. Aucun algorithme n'occupe simultanément les positions dominantes en débit, en portabilité et en avalanche.

Cette configuration résulte de mécanismes distincts, AES tire parti de l'accélération matérielle, ChaCha20 bénéficie d'un design ARX plus homogène entre architectures, tandis que DES, 3DES et Twofish restent contraints par des implémentations logicielles moins favorisées sur le matériel actuel. La décision finale doit donc demeurer contextuelle et reposer sur une pondération explicite des critères prioritaires du cas d'usage.

Pour prolonger cette lecture synthétique, le graphique suivant propose une carte de chaleur multicritère. Chaque algorithme y est croisé avec trois dimensions normalisées, le débit, l'efficacité en latence et la robustesse d'avalanche.

Graphique 10 — Carte de chaleur multicritère par algorithme



Le dixième graphique confirme, pour les dimensions qu'il couvre, la même hiérarchie que le radar entre débit, latence et diffusion, AES dominant nettement le débit et les cinq algorithmes convergeant vers une robustesse de diffusion quasi équivalente. La portabilité inter-plateforme, mise en évidence par le radar comme un facteur qui renverse le classement pour DES et 3DES, n'est pas reprise dans cette carte de chaleur. Dans cette

visualisation, l'efficacité en latence correspond à une mesure locale de temps de chiffrement normalisé, et non à un critère de stabilité inter-plateforme. Les deux représentations demeurent donc complémentaires plutôt que strictement redondantes.

Synthèse

Ce projet visait à comparer expérimentalement cinq algorithmes de chiffrement symétrique, AES, DES, 3DES, Twofish et ChaCha20, sur deux plateformes matérielles aux profils distincts, afin de dépasser une évaluation strictement théorique de la sécurité et d'intégrer la contrainte matérielle réelle. Les résultats confirment que le choix d'un algorithme ne peut pas reposer uniquement sur sa robustesse cryptographique, la performance observée dépendant fortement de la présence ou de l'absence d'accélération matérielle dédiée.

La validation fonctionnelle par KAT a confirmé la conformité de l'ensemble des implémentations aux références normatives retenues, avec 60 assertions sur 60 réussies en profil principal et 2268 sur 2268 en profil complet. Les mesures de performance montrent qu'AES domine nettement sur x86 grâce à AES-NI, avec un écart pouvant atteindre 6,70 fois par rapport au Raspberry Pi dans les conditions de débit maximal. À l'inverse, ChaCha20 présente le meilleur compromis de portabilité entre les deux plateformes, avec un ratio x86/Pi d'environ 1,40 dans la comparaison à 4096 octets. L'analyse de diffusion confirme par ailleurs une cohérence globale avec l'attendu théorique, les scores d'avalanche restant proches de 50 % sur les deux plateformes.

Deux résultats en particulier ne se contentent pas de rejoindre la littérature de façon générale, ils confirment une prédiction explicite formulée par la source elle-même, la stabilité inter-plateforme de ChaCha20 par rapport à la justification de conception de la RFC 8439 (Nir et Langley, 2018), et la convergence vers 50 % de l'effet d'avalanche par rapport au critère de diffusion stricte de Stallings (2017) et au cadre confusion-diffusion de Katz et Lindell (2020). Cette concordance demeure qualitative plutôt que quantitative stricte, les valeurs absolues dépendant du matériel, des bibliothèques et du protocole expérimental

propres à ce projet, mais elle confirme que les mécanismes structurels décrits par la littérature se manifestent bel et bien dans les données mesurées.

La confrontation systématique des résultats aux sources normatives a également permis d'identifier deux artefacts méthodologiques initialement masqués par des mesures en apparence cohérentes, l'un dans le protocole de mesure d'avalanche de ChaCha20 et l'autre dans la sélection des bits de clé pour DES et 3DES. Ces deux points ont été corrigés avant la consolidation finale des résultats.

Validation des hypothèses initiales

Les hypothèses formulées au démarrage du projet sont globalement confirmées par les données finales. Les constats principaux sont les suivants.

- La conformité fonctionnelle est validée avant l'analyse de performance, avec des KAT réussis sans échec sur les deux profils, 60 sur 60 et 2268 sur 2268.
- L'effet de la plateforme matérielle est confirmé, avec des écarts x86 Raspberry Pi marqués, en particulier pour AES avec accélération AES-NI.
- L'hypothèse de portabilité relative de ChaCha20 est confirmée, son écart interplateforme restant plus contenu que celui d'AES dans la comparaison à 4096 octets.
- L'hypothèse sur l'avalanche est validée dans le cadre du protocole retenu, les scores demeurant proches de la cible théorique de 50 % et globalement stables entre plateformes.

En synthèse, les hypothèses initiales sont validées. Une nuance méthodologique demeure essentielle, l'interprétation des résultats doit distinguer explicitement l'effet algorithmique de l'effet plateforme, comme l'illustre la contraction du ratio x86/Pi d'AES entre le débit maximal (6,70×) et la comparaison à taille fixe de 4096 octets (3,24×).

Recommandations selon le contexte de déploiement

Ces résultats conduisent à formuler des recommandations différenciées selon le contexte de déploiement, plutôt qu'à retenir un choix unique d'algorithme.

Tableau 4 — Recommandations d'algorithme selon le contexte de déploiement

Contexte	Algorithme recommandé	Justification	Référence
Production générale	AES-256-GCM	Compromis robuste entre sécurité, standardisation et déploiement industriel	NIST SP 800-38D
Systèmes embarqués et IoT (Internet of Things)	ChaCha20	Efficacité logicielle élevée sans AES-NI et bonne portabilité x86/ARM	RFC 8439
Référence et audit des algorithmes obsolètes	Éviter DES et 3DES	Maintien uniquement pour contraintes strictes de compatibilité	NIST SP 800-131A Rev. 2

Ce tableau traduit directement les écarts mesurés entre les deux plateformes. AES demeure le choix de référence lorsque le matériel cible dispose d'une accélération dédiée telle qu'AES-NI. En l'absence de ce support, ChaCha20 devient généralement préférable en raison d'un écart interplateforme plus contenu. Quant à DES et 3DES, leur maintien ne se justifie plus que par des impératifs de compatibilité avec des systèmes existants. Cette position est cohérente avec la trajectoire de transition NIST, où SP 800-131A encadre l'usage résiduel des algorithmes obsolètes.

Perspectives et risques émergents

Au-delà du protocole expérimental, deux risques émergents doivent être pris en compte pour apprécier la pérennité des recommandations précédentes, le risque quantique et le risque lié à l'intelligence artificielle.

Le risque quantique réduit principalement la marge théorique de sécurité, sans démonstration à ce jour d'une rupture pratique d'AES-256 dans les conditions d'usage courantes. La réponse opérationnelle n'est donc pas un remplacement immédiat d'AES-256-GCM, mais la préparation d'une stratégie de crypto-agilité et d'une trajectoire post-quantique, en priorité pour les mécanismes asymétriques, historiquement plus exposés aux accélérations quantiques connues (NIST SP 800-57 Part 1 Rev. 5, 2020).

Selon les référentiels NIST retenus, le risque associé à l'intelligence artificielle est aujourd'hui majoritairement opérationnel plutôt qu'algorithmique. Dans ce cadre, l'intelligence artificielle amplifie surtout des risques déjà documentés de gouvernance et d'exploitation, notamment des erreurs de configuration, la compromission de secrets et l'automatisation d'attaques, davantage qu'elle ne produit une cryptanalyse nouvelle contre AES-256 ou ChaCha20. La priorité demeure donc le durcissement opérationnel, incluant la gouvernance des clés, la supervision des configurations et la réduction de la surface d'attaque humaine et logicielle (NIST AI RMF 1.0, 2023; NIST AI 600-1, 2024).

Ces deux risques convergent vers une même conclusion pratique, la nécessité d'instaurer une crypto-agilité effective. Cela implique de maintenir un inventaire cryptographique à jour, de préparer des modes de transition hybrides, d'assurer une rotation régulière des clés et de définir une trajectoire explicite vers des mécanismes post-quantiques pour la composante asymétrique des systèmes (NIST SP 800-57 Part 1 Rev. 5, 2020; NIST AI RMF 1.0, 2023; NIST AI 600-1, 2024).

Dans la phase finale du projet, cette orientation a déjà été concrétisée dans le protocole retenu, où la validation fonctionnelle et la stabilité statistique reposent sur une chaîne d'intégration continue automatisée, décrite à la section Validation fonctionnelle (KAT). L'intérêt pour la crypto-agilité est direct, ces contrôles réduisent le risque de régression silencieuse, accélèrent la détection d'écarts de conformité et maintiennent une base vérifiable pour faire évoluer les choix cryptographiques sans perdre la maîtrise des risques.

Limites de l'étude

Le périmètre retenu privilégie trois objectifs, la conformité fonctionnelle, la performance et la diffusion cryptographique, dans un cadre strictement reproductible. Dans cette logique, la consommation énergétique et la résistance aux canaux auxiliaires n'ont pas été instrumentées dans la campagne principale, même si ces dimensions restent déterminantes pour une mise en production.

Par ailleurs, l'analyse de la convergence de l'effet d'avalanche selon le nombre de tours internes n'a pas été traitée comme un axe quantitatif complet dans cette itération. Elle constitue une extension méthodologique prioritaire pour un cycle expérimental ultérieur, afin d'affiner encore l'interprétation des propriétés de diffusion.

Conclusion générale

Ce rapport démontre qu'une comparaison crédible des algorithmes de chiffrement symétrique exige une approche intégrée, combinant validation fonctionnelle, protocole expérimental contrôlé, analyse statistique et interprétation contextualisée des résultats. Les données obtenues confirment que les performances observées ne dépendent pas uniquement de la conception cryptographique, mais aussi de l'adéquation entre l'algorithme, le mode d'opération et l'architecture matérielle d'exécution.

Sur le plan applicatif, les résultats soutiennent un positionnement différencié selon le contexte de déploiement. AES-GCM demeure la solution de référence dans les environnements disposant d'accélération matérielle adaptée, tandis que ChaCha20 constitue une option particulièrement pertinente lorsque cette accélération est absente et que la portabilité interplateforme devient prioritaire. Les algorithmes DES et 3DES ne conservent un intérêt que pour des besoins stricts de compatibilité avec des systèmes existants.

Au total, la contribution principale de ce travail réside dans l'établissement d'un cadre de comparaison reproductible et traçable, directement exploitable pour appuyer des décisions techniques fondées sur des mesures. Ce cadre fournit une base méthodologique robuste pour les évolutions futures, notamment l'intégration progressive d'exigences de crypto-agilité et l'adaptation aux transformations du paysage de sécurité.

Références

Katz, J., & Lindell, Y. (2020). *Introduction to Modern Cryptography* (3rd ed.). CRC Press.

National Institute of Standards and Technology. (1998). *NIST SP 800-17: Modes of Operation Validation System (MOVS): Requirements and Procedures*. <https://csrc.nist.gov/pubs/sp/800/17/final>

- National Institute of Standards and Technology. (1999). *FIPS 46-3: Data Encryption Standard (DES)*.
<https://csrc.nist.gov/pubs/fips/46-3/final>
- National Institute of Standards and Technology. (2001). *FIPS 197: Advanced Encryption Standard (AES)*.
<https://csrc.nist.gov/pubs/fips/197/final>
- National Institute of Standards and Technology. (2001a). *NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation*. <https://csrc.nist.gov/pubs/sp/800/38/a/final>
- National Institute of Standards and Technology. (2007). *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: GCM and GMAC*. <https://csrc.nist.gov/pubs/sp/800/38/d/final>
- National Institute of Standards and Technology. (2017). *NIST SP 800-67 Rev. 2: Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher*.
<https://csrc.nist.gov/pubs/sp/800/67/r2/final>
- National Institute of Standards and Technology. (2019). *NIST SP 800-131A Rev. 2: Transitioning the Use of Cryptographic Algorithms and Key Lengths*. <https://doi.org/10.6028/NIST.SP.800-131Ar2>
- National Institute of Standards and Technology. (2020). *NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management*. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- National Institute of Standards and Technology. (2023). *NIST AI Risk Management Framework (AI RMF 1.0)*. <https://doi.org/10.6028/NIST.AI.100-1>
- National Institute of Standards and Technology. (2024). *NIST AI 600-1: Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*.
<https://doi.org/10.6028/NIST.AI.600-1>
- Nir, Y., & Langley, A. (2018). *RFC 8439: ChaCha20 and Poly1305 for IETF Protocols*. <https://www.rfc-editor.org/rfc/rfc8439>
- Schneier, B., Kelsey, J., Whiting, D., Wagner, D., Hall, C., & Ferguson, N. (n.d.). *Twofish Known-Answer Test Vectors*. <https://www.schneier.com/academic/twofish/download/>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.

Annexes

Annexe A – Code source

Le code source Python complet est joint à ce rapport. Son organisation suit la séparation des responsabilités présentée dans l'architecture du système.

- Domaine, primitives cryptographiques (domain/cipher/) : CipherPrimitive.py (interface), AES.py, DES.py, TripleDES.py, Twofish.py, ChaCha20.py.
- Domaine, modes d'opération (domain/mode/) : OperationMode.py (interface), ECB.py, CBC.py, CTR.py, GCM.py, StreamMode.py.
- Domaine, moteur de chiffrement (domain/engine/) : EncryptionEngine.py (composition d'une primitive et d'un mode).
- Couche application (application/) : ExperimentController.py (orchestration des mesures de performance, avalanche et avalanche clé).
- Validation KAT (validation/) : kat_aes.py, kat_des.py, kat_3des.py, kat_modes.py, kat_gcm.py, kat_chacha20.py, kat_twofish.py.
- Scripts d'exécution et d'audit (scripts/) : experiment.py (campagne expérimentale), run_kat.py (validation KAT), run_charts.py (génération des graphiques), audit/ (audit IC95 et dossier de preuves).

Le détail exhaustif des modules est fourni dans les fichiers joints du projet.

Annexe B – Liste des acronymes

- 3DES : Triple Data Encryption Standard
- AES : Advanced Encryption Standard (Norme de chiffrement avancé)
- AES-NI : Advanced Encryption Standard New Instructions (Instructions matérielles AES)
- AI : Artificial Intelligence (Intelligence artificielle)
- AI RMF : AI Risk Management Framework (Cadre de gestion des risques en intelligence artificielle)
- ARM : Advanced RISC Machine (Architecture RISC avancée)
- ARX : Addition-Rotation-XOR
- CBC : Cipher Block Chaining (Chaîne de blocs chiffrés)

- CI : Continuous Integration (Intégration continue)
- CSV : Comma-Separated Values (Valeurs séparées par des virgules)
- CTR : Counter Mode (Mode compteur)
- DES : Data Encryption Standard (Norme de chiffrement des données)
- ECB : Electronic Codebook (Livre de codes électronique)
- FIPS : Federal Information Processing Standards (Normes fédérales de traitement de l'information)
- GCM : Galois Counter Mode (Mode Galois compteur)
- IC95 : 95 % confidence interval (Intervalle de confiance à 95 %)
- IoT : Internet of Things (Internet des objets)
- IV : Initialization Vector (Vecteur d'initialisation)
- KAT : Known Answer Tests (Tests à réponse connue)
- MB/s : Megabytes per second (Mégaoctets par seconde)
- NIST : National Institute of Standards and Technology (Institut national des normes et de la technologie)
- RFC : Request for Comments (Demande de commentaires)
- SDLC : Software Development Life Cycle (Cycle de vie du développement logiciel)
- SHA-256 : Secure Hash Algorithm 256-bit (Algorithme de hachage sécurisé 256 bits)
- SP : Special Publication (Publication spéciale)
- x86 : x86 processor architecture (Architecture de processeur x86)